TEAM 1

GECT Connect

User Registration & Authentication Module

Mariya Shibi Project Coordination & Integration

Libiya Sebu Login Screen, Session Mgmt, User DAO

Jishnu S Dashboard, AUTH Service, DB Connection

Jayasurya Madhav Jayn Registration Screen, Forgot Password

Project Introduction

What is GECT Connect?

A WhatsApp-like campus messaging platform exclusively for students and staff of Government Engineering College Thrissur. Built with Java Swing + MySQL, it enables secure, department-aware communication across 7 integrated modules.

7 Integrated Modules

1. Authentication
2. User Profile
3. Contacts
4. Individual Chat
5. Group Chat
6. Notifications
7. Settings & Feed

Tech Stack:

- Frontend: Java Swing / JavaFX
- Backend: MySQL Database
- Connectivity: JDBC
- Pattern: MVC + DAO
- OOP: Inheritance, Polymorphism, Encapsulation

"Connecting the GECT community — one message at a time."

Team 1 Objectives

User Registration & Authentication Module

01

Secure Registration

Allow students and staff to create accounts using verified @gectcr.ac.in email addresses with role-specific fields (Roll No for students, Emp ID for staff).

02

Authentication System

Implement a robust login mechanism that validates credentials against the MySQL database using PreparedStatements to prevent SQL injection.

03

OTP Verification

Simulate email-based OTP for account verification and password reset workflows to demonstrate real-world security practices.

04

OOP Architecture

Apply Inheritance (User → Student/Staff), Encapsulation (private fields + getters/setters), and Constructor Overloading across the module.

05

Clean Layer Separation

Follow Model → DAO → Service → UI layered architecture for maintainability and seamless integration with other teams' modules.

06

Session Management

Maintain secure user sessions post-login, providing user context (userId, role, department) for downstream module operations.

Functional & Technical Requirements

Functional Requirements

- Register with @gectcr.ac.in email (unique, validated)
- Role-based fields: Roll No (Student), Emp ID (Staff)
- Full name, department, mobile number (all required)
- Login using college email + password
- OTP simulation for email verification
- Forgot password → OTP → reset new password
- Dashboard screen after successful login
- Real-time validation feedback (empty fields, format)
- Password minimum length ≥ 6 characters

OOP Concepts & DB Operations

Inheritance: User (base) → Student, Staff (derived)

Encapsulation: Private fields with public getters/setters

Constructor Overloading: Student(email, name, roll) vs Staff(email, name, empId)

Method Overriding: toString(), validate() overridden in subclasses

DB INSERT: INSERT INTO users with PreparedStatement

DB SELECT: SELECT for login credential validation

DB UPDATE: UPDATE password after OTP verification

DB UNIQUE check: Email uniqueness before registration

GUI Screens — Requirements Detail

Screen 1

Login Screen

Email field (placeholder: @gectcr.ac.in)
Password field (show/hide toggle)
Login button with loading indicator
Forgot Password link

✓ Email domain check + min password length

Screen 2

Registration Form

Name, Email, Dept dropdowns
Role selector (Student/Staff)
Roll No / Employee ID (dynamic)
Password + Confirm Password

✓ All fields required + email unique check

Screen 3

OTP Verification

6-digit OTP input field
Display simulated OTP
Verify button enables on match
Resend OTP option

✓ OTP exact match required

Screen 4

Forgot Password

Email input for account lookup
Generate & display OTP
New password + confirm fields
Update Password button

✓ OTP + new password ≥ 6 chars

Screen 5

Dashboard

Welcome message with user name
Role badge (Student / Staff)
Department & ID display
Navigation to all 7 modules

✓ Session required — redirect if not logged in

Detailed Requirements — Part 1 of 2

Functional Requirements

ID	Feature	Key Requirements
FR-01	User Registration	College email (@gectcr.ac.in), Full name, Dept (CS/ECE/ME/CE/EEE), Role (Student/Staff), Roll No. or Emp ID, Mobile (10-digit), Password (min 6 chars), Confirm Password
FR-02	Login Authentication	Validate email format + credentials against DB; store User in SessionManager; update last_seen; navigate to Dashboard
FR-03	OTP Verification	Generate 6-digit OTP; save to otp_records table with 10-min expiry; show OTP in dialog (simulated); verify entry
FR-04	Forgot Password	Step 1: Email lookup; Step 2: OTP verify; Step 3: New password entry + confirm — all on one screen using CardLayout
FR-05	Dashboard	Welcome header with name/role/dept; 3x2 grid of module buttons for Teams 2-7; Logout clears session → Login screen

Detailed Requirements — Part 2 of 2

GUI Screens & OOP Concepts

GUI Screens (5 screens — Java Swing)

S1	**LoginScreen.java**	Libiya Sebu
S2	**RegistrationScreen.java**	Jayasurya Madhav Jayan
S3	**OTPVerificationScreen.java**	Jishnu S
S4	**ForgotPasswordScreen.java**	Maria Shibi
S5	**DashboardScreen.java**	Jishnu S

OOP Concepts Demonstrated

1. **Inheritance**
   User (abstract) → Student, Staff
2. **Encapsulation**
   Private fields + getters/setters in all models
3. **Abstraction**
   Abstract User class with getDisplayInfo()
4. **Method Overriding**
   Student & Staff override getDisplayInfo()
5. **Constructor Overload**
   Registration vs DB-load constructors
6. **Singleton Pattern**
   SessionManager & DBConnection

Design — Class Diagram

OOP Architecture

<<abstract>> User

- userId : int
- email : String
- passwordHash : String
- fullName : String
- role : String
- department : String

+ getDisplayInfo() : String {abstract}
+ getUserId() / getEmail() / ...getters

Abstract Class
Concrete subclass
DAO layer
Service layer
Session layer

UserDAO

+ registerUser(User) : int
+ getUserByEmail(String) : User
+ emailExists(String) : boolean
+ updatePassword(int, String)
+ updateLastSeen(int)
+ saveOTP(int, String, String)

AuthService

+ login(email, pass) : User
+ register(User) : boolean
+ changePassword(...)

Student extends User

- rollNumber : String

+ Student(email, pass, ..., rollNo)
+ getDisplayInfo() : String {override}

Staff extends User

- employeeId : String

+ Staff(email, pass, ..., empId)
+ getDisplayInfo() : String {override}

SessionManager <<singleton>>

+ getCurrentUser() : User
+ setCurrentUser(User)

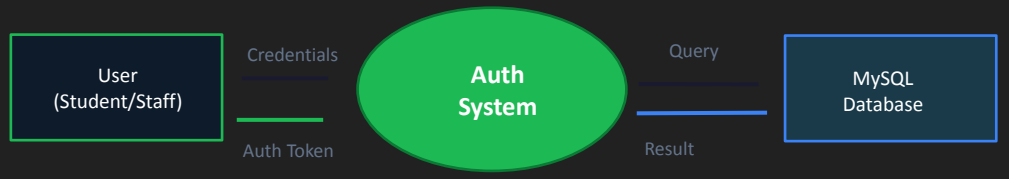
Class Diagram

User Registration & Authentication Module

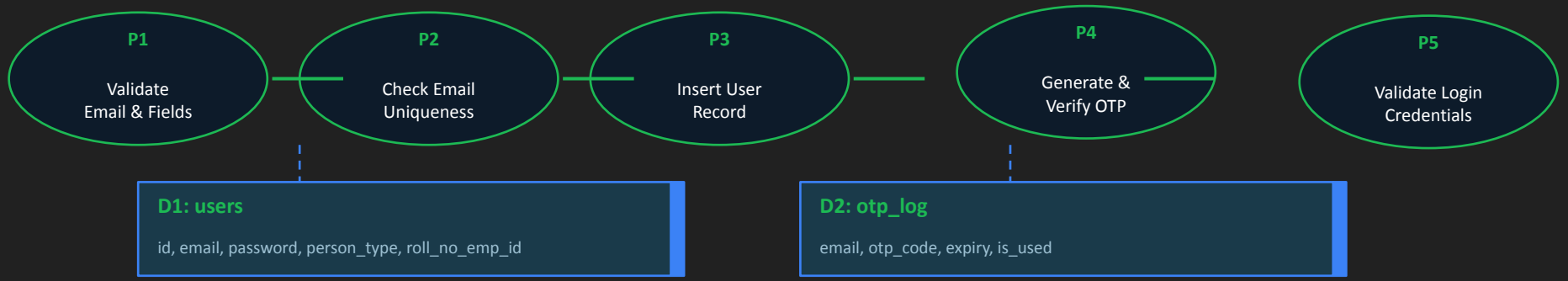
Data Flow Diagram

Level 0 (Context) + Level 1 (Process Breakdown)

Level 0 — Context Diagram



Level 1 — Process Breakdown



Database Schema

MySQL + JDBC • Normalization: 3NF

TABLE: users

Column	Type	Constraints
id	INT	PRIMARY KEY, AUTO_INCREMENT
email	VARCHAR(255)	UNIQUE, NOT NULL, CHECK @gectcr.ac.in
full_name	VARCHAR(255)	NOT NULL
password	VARCHAR(255)	NOT NULL, length ≥ 6
person_type	ENUM	'STUDENT', 'STAFF' — NOT NULL
roll_no_emp_id	VARCHAR(50)	UNIQUE, NOT NULL
department	VARCHAR(100)	NULL allowed
mobile	VARCHAR(15)	NULL allowed
is_online	BOOLEAN	DEFAULT FALSE
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Normalization

✓ **1NF**

All attributes are atomic (no multi-valued). Each column stores a single value.

✓ **2NF**

No partial dependencies. All non-key attrs depend on full primary key (id).

✓ **3NF**

No transitive dependencies. dept/mobile don't determine each other.

★ **BCNF**

email is a candidate key — table is in BCNF as well.

✓ **Schema achieves 3NF (and BCNF)**

GUI Screenshots

Java Swing Implementation

GECT Connect

[Insert Screenshot Here]

Login Screen

Email + Password + Validation

GECT Connect

[Insert Screenshot Here]

Registration Form

Role-based fields + OTP

GECT Connect

[Insert Screenshot Here]

OTP Verification

6-digit simulated OTP

GECT Connect

[Insert Screenshot Here]

Forgot Password

Email + OTP + Reset

GECT Connect

[Insert Screenshot Here]

Dashboard

Post-login navigation hub

GUI Screenshots

Test Log

Authentication Module — Standard Format

Test ID	Test Case	Input	Expected Output	Actual Output	Status
TC-01	Valid Registration	Valid @gectcr.ac.in email + all fields	Account created, OTP sent	Account created successfully	**PASS**
TC-02	Invalid Email Domain	user@gmail.com	Error: Invalid email domain	Error message shown	**PASS**
TC-03	Duplicate Email	Existing @gectcr.ac.in email	Error: Email already registered	Error message shown	**PASS**
TC-04	Short Password	Pass: '123'	Error: Min 6 chars required	Validation error shown	**PASS**
TC-05	Empty Fields	Blank name, dept	Error: All fields required	All fields validated	**PASS**
TC-06	Valid Login	Correct email + password	Redirect to Dashboard	Dashboard loaded	**PASS**
TC-07	Wrong Password	Correct email + wrong password	Error: Invalid password	Error shown, no login	**PASS**
TC-08	OTP Verification	Correct 6-digit OTP entered	OTP verified, proceed	Register button enabled	**PASS**
TC-09	Wrong OTP	Incorrect OTP entered	Register button stays disabled	Button remains disabled	**PASS**
TC-10	Password Reset	Valid email + OTP + new password ≥ 6 chars	Password updated, login with new	Password updated successfully	**PASS**
TC-11	Session Persistence	Login → navigate modules	User session maintained	UserId available in SessionMgr	**PASS**
TC-12	Logout	Click logout button	Session cleared, redirect to login	Login screen shown	**PASS**

Team Contributions

Mariya Shibi

Project Coordination & Integration

- Defined overall module architecture & package structure
- Designed EventBus integration between Auth and MainFrame
- Coordinated team deliverables and code review
- Implemented integration points with teams 2-7

🎤 *Introduces project, demo integration flow*

Libiya Sebu

Login Screen, Session Mgmt, User DAO

- Built LoginPage.java with password show/hide toggle
- Implemented AuthDAO.authenticate() using PreparedStatement
- Coded AuthDAO.isEmailExists() for validation
- Implemented AuthDAO.updateOnlineStatus() on login/logout

🎤 *Demo Login + Session flow + DAO queries*

Jishnu S

Dashboard, AUTH Service, DB Connection

- Built Dashboard screen with navigation to all 7 modules
- Implemented AuthService.login() & AuthService.register()
- Configured DBConnection.java (JDBC, MySQL driver loading)
- Implemented email domain validation in AuthService

🎤 *Demo Dashboard + Service layer + DB connection*

Jayasurya Madhav Jayn

Registration Screen, Forgot Password

- Built RegisterPage.java with role-based dynamic fields
- Implemented OTP generation and verification logic
- Built ForgotPasswordPage.java with step-wise flow
- Coded User, Student, Staff model classes (OOP hierarchy)

🎤 *Demo Register + Forgot Password + OOP hierarchy*

Thank You

Questions & Answers

Module: User Registration & Authentication

Team 1 | GEC Thrissur | MCA Micro Project 2025

Mariya Shibi
Coordination

Libiya Sebu
Login / DAO

Jishnu S
Dashboard / DB

Jayasurya
Register / OTP

Government Engineering College Thrissur